Day 3

Wednesday, 3 June 2026 9:21 AM

198. House Robber

Solved

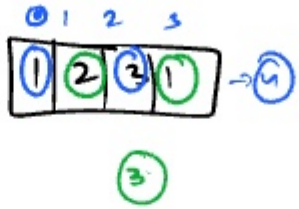
Medium | Topics | Companies

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night.**

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police.***

Example 1:

```
Input: nums = [1,2,3,1]
Output: 4
```



↳ Rob the current house + skip the next

↳ Skip the current house + rob the next house

→ Rob → current house + skip the next.

→ Skip the current house & start robbing from next house.

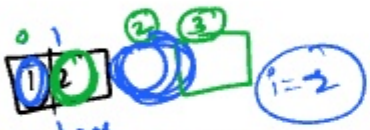
Robbing current house → 10 + 14

Skipping current house

2+14 , 5

26

24 16



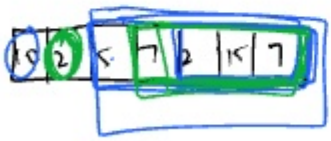
i = 2

Rob current house

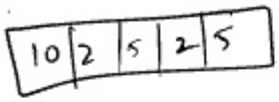
↳ nums[i] + helper(i+2)

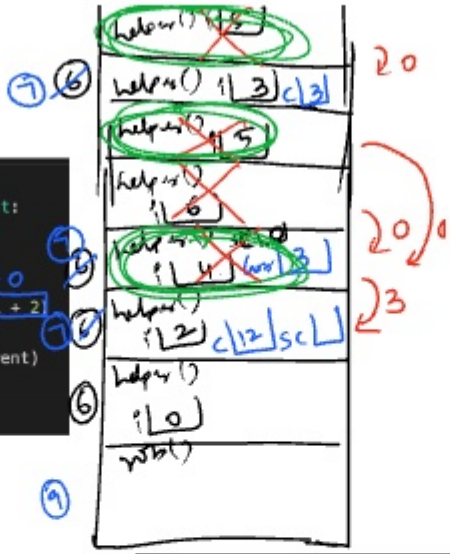
Skip the current house

↳ helper(i+1)



helper() : 4





```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i) :
            if i >= len(nums) :
                return 0
            current = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            return max(current, skipCurrent)
        return helper(0)
```

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i) :
            if i >= len(nums) :
                return 0
            current = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            return max(current, skipCurrent)
        return helper(0)
```



```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i, dp) :
            if i >= len(nums) :
                return 0
            if dp[i] != -1 :
                return dp[i]
            current = nums[i] + helper(i + 2, dp)
            skipCurrent = helper(i + 1, dp)
            dp[i] = max(current, skipCurrent)
            return dp[i]

        dp = [-1] * len(nums)
        # dp = []
        # for i in range(len(nums)) :
        #     dp.append(-1)
        return helper(0, dp)
```

RECURSIVE DP :

→ P No: 198

→ T.C → O(n)

→ S.C → O(n)

n + n → 2n

↓ ↓

size dp stackspace

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        if len(nums) == 0 :
            return 0
        if len(nums) == 1 :
            return nums[0]
        if len(nums) == 2 :
            return max(nums[0], nums[1])
```

→ Iterative DP

→ T.C → O(n)

```
        dp = []
        dp.append(nums[0])
        dp.append(max(nums[0], nums[1]))
        for i in range(2, len(nums)) :
            current = nums[i] + dp[i - 2]
            skipCurrent = dp[i - 1]
            dp.append(max(current, skipCurrent))
        return dp[-1]
```

→ S C → O(n)

↓

Size of dp array

746. Min Cost Climbing Stairs

Easy · Topics · Companies · Hint

You are given an integer array `cost` where `cost[i]` is the cost of `i`th step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

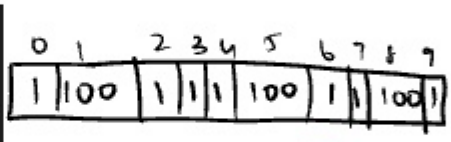
Return *the minimum cost to reach the top of the floor.*

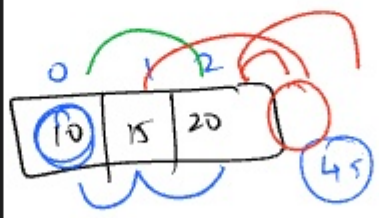
Example 1:

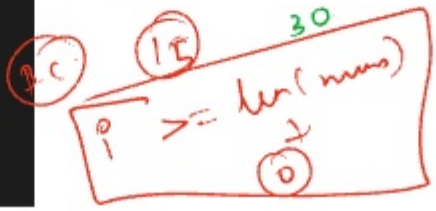
```
Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.
```

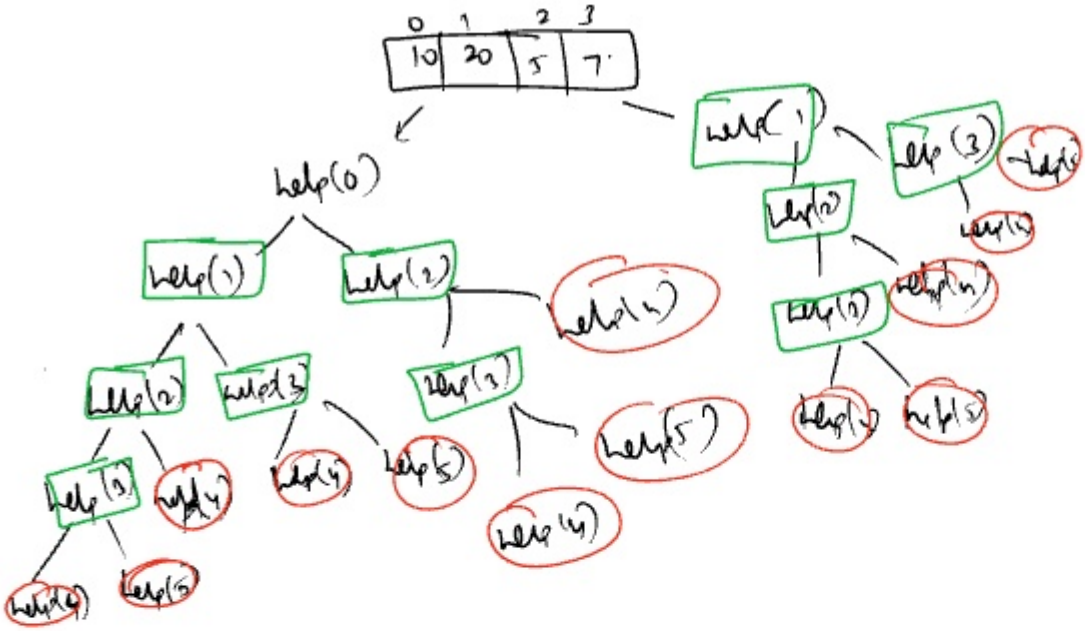
Example 2:

```
Input: cost = [1,100,1,1,1,100,1,1,100,1]
Output: 6
```










```
class Solution:
    def minCostClimbingStairs(self, cost: List[int])
    -> int:
        def helper(i) :
            if i >= len(cost) :
                return 0
            ans1 = helper(i + 1)
            ans2 = helper(i + 2)
            return cost[i] + min(ans1, ans2)
        return min(helper(0), helper(1))
```

→ Recursive Approach

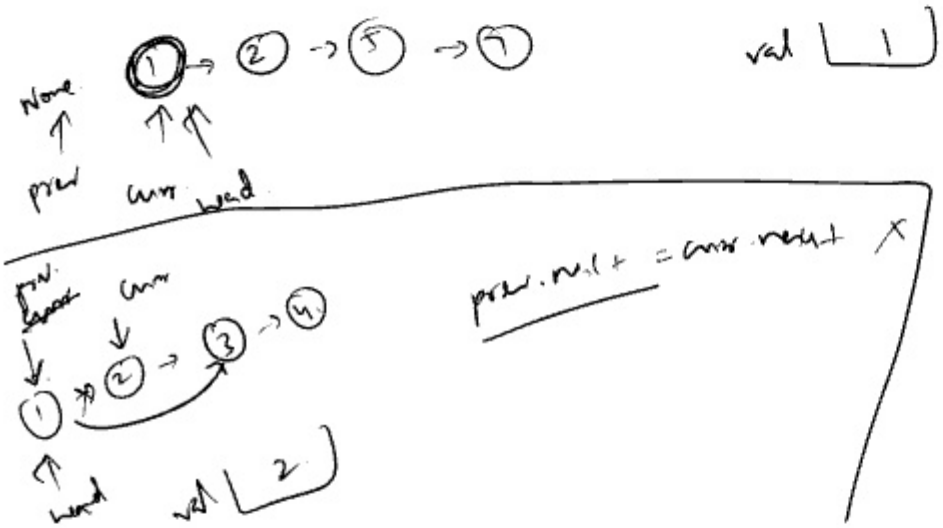
```python
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        dp = [-1] * len(cost)
        def helper(i) :
            if i >= len(cost) :
                return 0
            if dp[i] != -1 :
                return dp[i]
            ans1 = helper(i + 1)
            ans2 = helper(i + 2)
            dp[i] = cost[i] + min(ans1, ans2)
            return dp[i]

        return min(helper(0), helper(1))
```

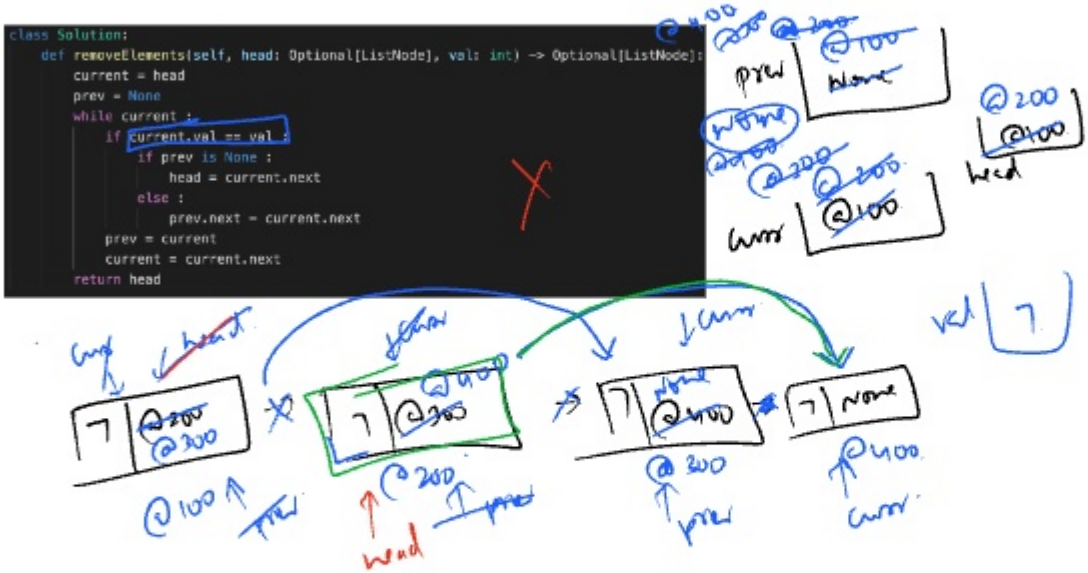
RECURSIVE DP

-> T.C -> O(n)

-> S.C -> O(n)



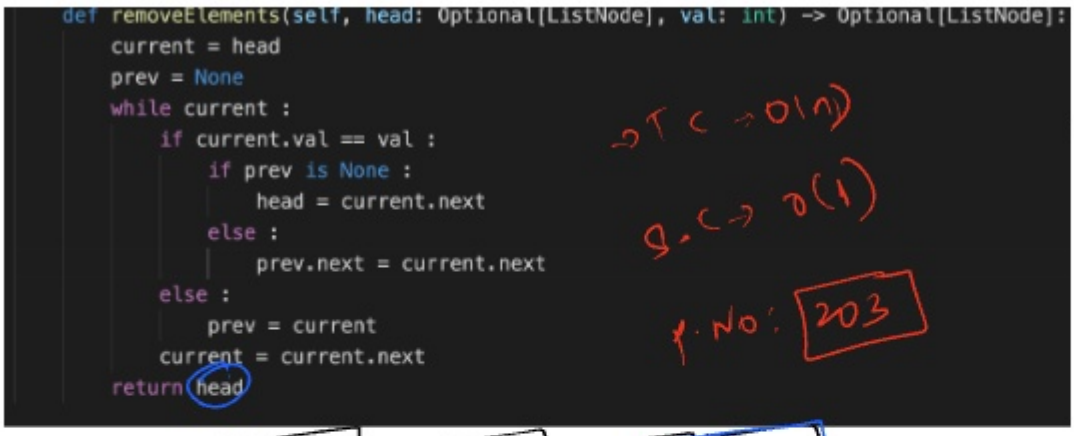

```python
class Solution:
    def removeElements(self, head: Optional[ListNode], val: int) -> Optional[ListNode]:
        current = head
        prev = None
        while current :
            if current.val == val :
                if prev is None :
                    head = current.next
                else :
                    prev.next = current.next
            prev = current
            current = current.next
        return head
```

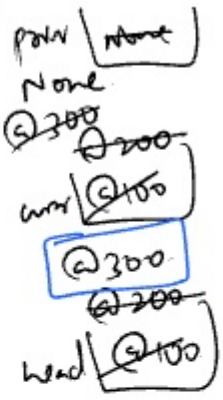


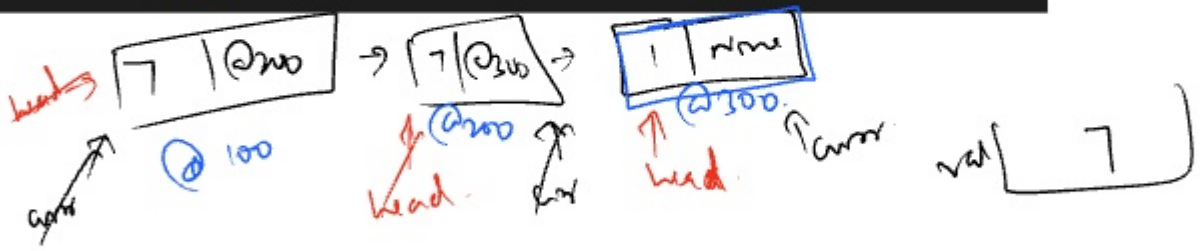

```python
class Solution:
```

@ 300

```python
def removeElements(self, head: Optional[ListNode], val: int) -> Optional[ListNode]:
    current = head
    prev = None
    while current :
        if current.val == val :
            if prev is None :
                head = current.next
            else :
                prev.next = current.next
        else :
            prev = current
        current = current.next
    return head
```







64. Minimum Path Sum

Solved

Medium · Topics · Companies

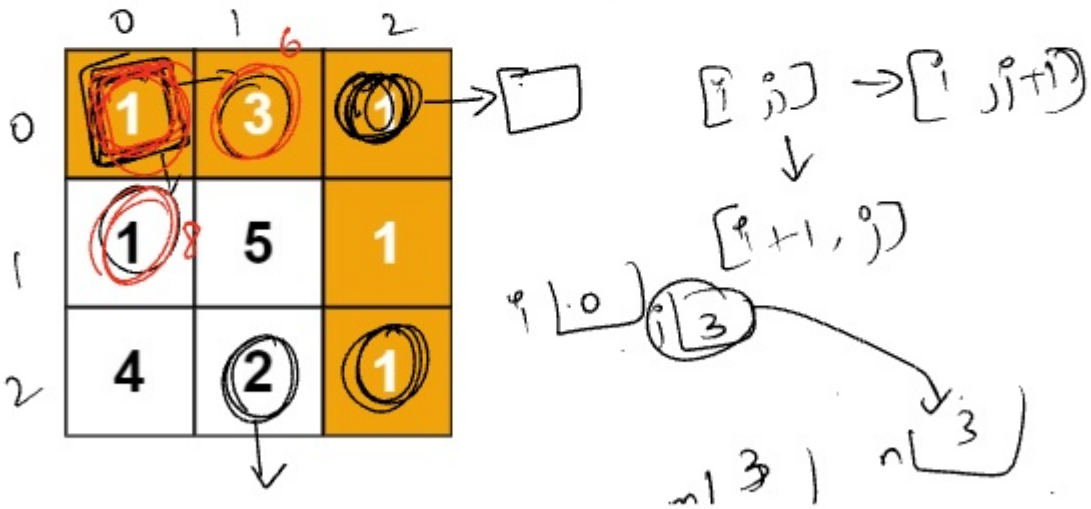
Given a `m x n grid` filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

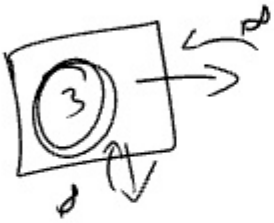
Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

```
Input: grid = [[1,3,1],[1,5,1],[4,2,1]]
Output: 7
```

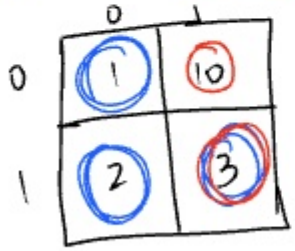


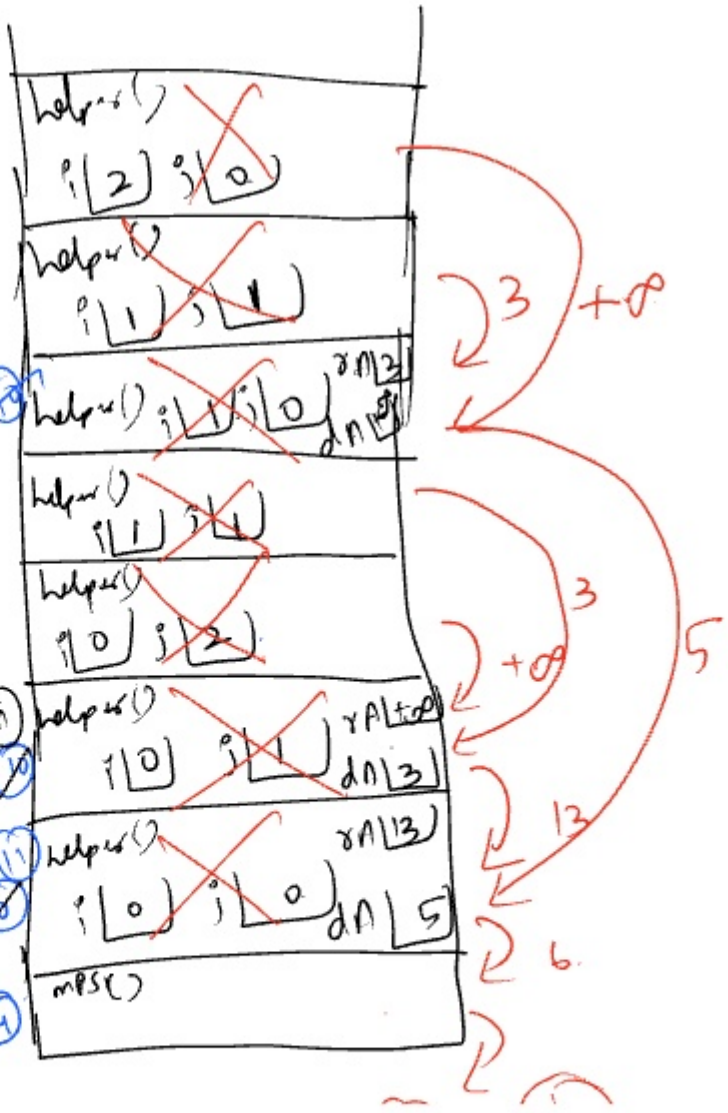


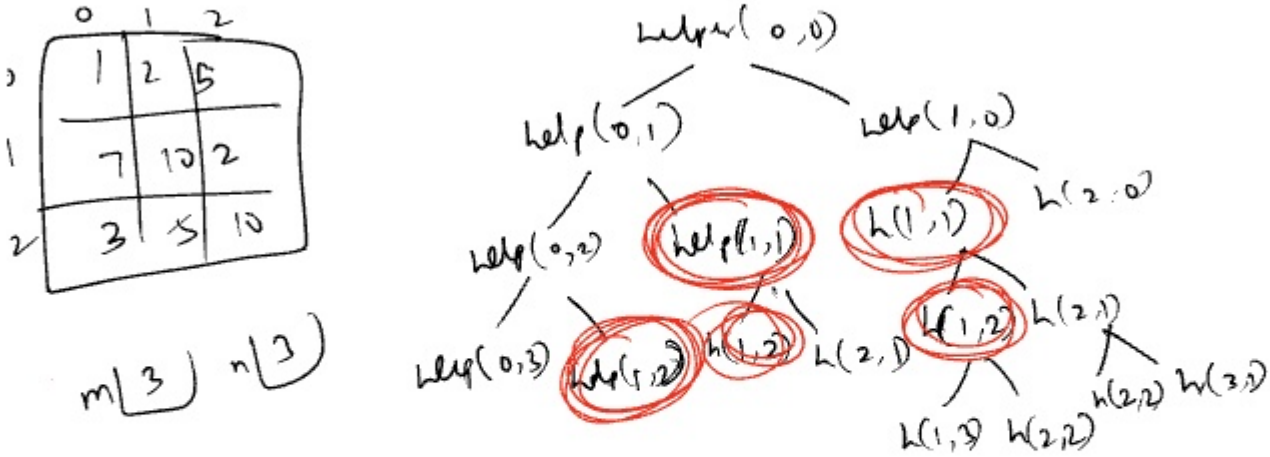
1	3	1
1	5	1
4	2	1

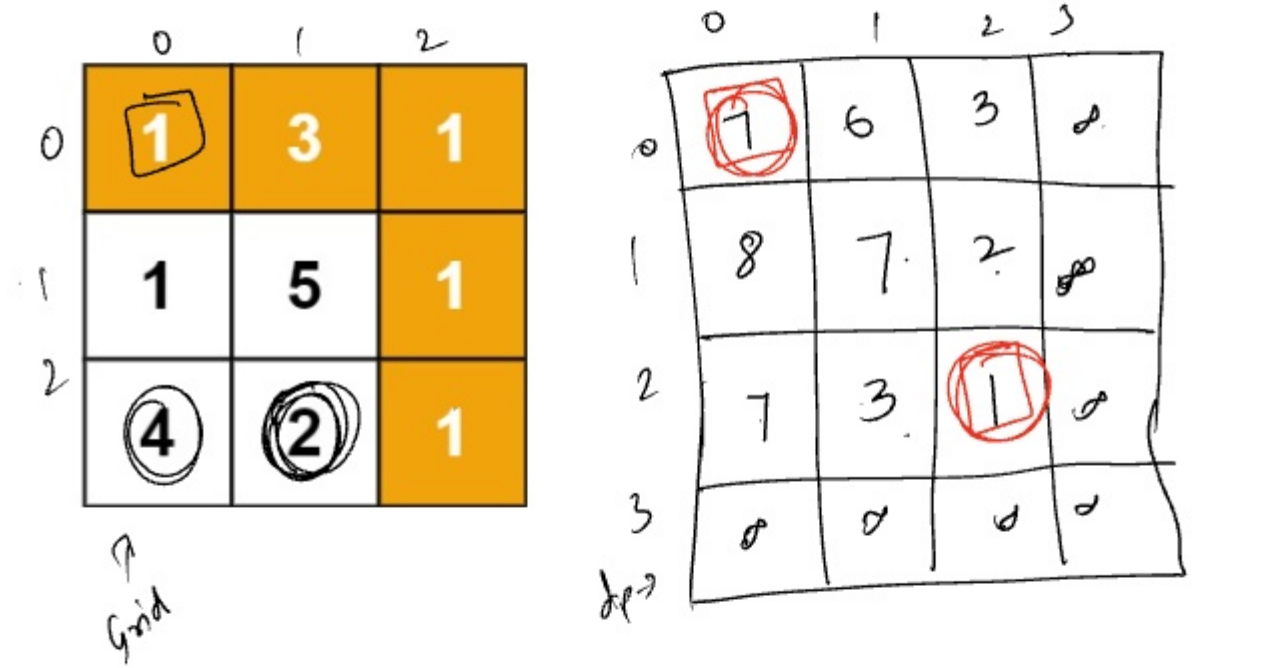
```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        def helper(i, j) :
            m = len(grid)
            n = len(grid[0])
            if i == m or j == n :
                return float('inf')
            if i == m - 1 and j == n - 1 :
                return grid[i][j]
            rightAns = helper(i, j + 1)
            downAns = helper(i + 1, j)
            return grid[i][j] + min(rightAns, downAns)

        return helper(0, 0)
```









```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        dp = []
        for i in range(len(grid) + 1) :
            current = []
            for j in range(len(grid[0]) + 1) :
                current.append(float('inf'))
            dp.append(current)
        m = len(grid)
        n = len(grid[0])
        for i in range(m - 1, -1, -1 ) :
            for j in range(n - 1, -1, -1) :
                if i == m - 1 and j == n - 1 :
                    dp[i][j] = grid[i][j]
```

→ T.C → O(m*n)

→ S.C → O(m*n)

```
                continue
            dp[i][j] = grid[i][j] + min(dp[i][j + 1], dp[i + 1][j])
        return dp[0][0]
```

165. Compare Version Numbers

Solved

Medium | Topics | Companies | Hint

Given two **version strings**, `version1` and `version2`, compare them. A version string consists of **revisions** separated by dots `'.'`. The **value of the revision** is its **integer conversion** ignoring leading zeros.

To compare version strings, compare their revision values in **left-to-right order**. If one of the version strings has fewer revisions, treat the missing revision values as `0`.

Return the following:

- If `version1 < version2`, return `-1`.
- If `version1 > version2`, return `1`.
- Otherwise, return `0`.

" 125.70.23.1 " ← V1

" 125.70.23.1 " ← V2

```
class Solution:
    def compareVersion(self, version1: str, version2: str) -> int:
        i = 0
        j = 0
        while i < len(version1) or j < len(version2):
            num1 = 0
            while i < len(version1) and version1[i] != '.' :
                num1 = (num1 * 10) + int(version1[i])
                i += 1
            num2 = 0
            while j < len(version2) and version2[j] != '.' :
                num2 = (num2 * 10) + int(version2[j])
                j += 1
            if num1 < num2 :
                return -1
            elif num1 > num2 :
                return 1
            i += 1
            j += 1
        return 0
```

→ P.No : 165

→ T.C → O(m+n)

→ S.C → O(1)